

Teste de integração entre camadas usando **Spring Boot + JPA + banco H2** 👍

Testar a integração entre:

- Service
- Repository
- Banco de dados

💡 Sem mock → integração real

Estrutura

```
com.exemplo.conta
├─ ContaApplication.java
├─ model/Conta.java
├─ repository/ContaRepository.java
├─ service/ContaService.java
└─ ContaServiceIntegrationTest.java
```

1. Entidade

```
package com.exemplo.conta.model;
```

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class Conta {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String titular;
```

```
    private Double saldo;
```

```
    public Conta() {}
```

```
    public Conta(String titular, Double saldo) {
```

```
        this.titular = titular;
```

```

        this.saldo = saldo;
    }

    public Long getId() { return id; }
    public String getTitular() { return titular; }
    public Double getSaldo() { return saldo; }

    public void setSaldo(Double saldo) { this.saldo = saldo; }
}

```

2. Repository

```

package com.exemplo.conta.repository;

import com.exemplo.conta.model.Conta;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ContaRepository extends JpaRepository<Conta, Long> {
}

```

3. Service

```

package com.exemplo.conta.service;

import com.exemplo.conta.model.Conta;
import com.exemplo.conta.repository.ContaRepository;
import org.springframework.stereotype.Service;

@Service
public class ContaService {

    private final ContaRepository repository;

    public ContaService(ContaRepository repository) {
        this.repository = repository;
    }

    public Conta salvar(Conta conta) {
        return repository.save(conta);
    }
}

```

```

    }

    public void transferir(Long origemId, Long destinoId, Double
valor) {

        Conta origem = repository.findById(origemId).orElseThrow();
        Conta destino = repository.findById(destinoId).orElseThrow();

        if (origem.getSaldo() < valor) {
            throw new RuntimeException("Saldo insuficiente");
        }

        origem.setSaldo(origem.getSaldo() - valor);
        destino.setSaldo(destino.getSaldo() + valor);

        repository.save(origem);
        repository.save(destino);
    }
}

```

4. Teste de Integração

```

package com.exemplo.conta;

import com.exemplo.conta.model.Conta;
import com.exemplo.conta.repository.ContaRepository;
import com.exemplo.conta.service.ContaService;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import jakarta.transaction.Transactional;

import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest

```

```
@Transactional
public class ContaServiceIntegrationTest {

    @Autowired
    private ContaService service;

    @Autowired
    private ContaRepository repository;

    @Test
    public void deveSalvarConta() {
        Conta conta = new Conta("João", 1000.0);

        Conta salva = service.salvar(conta);

        assertNotNull(salva.getId());
    }

    @Test
    public void deveTransferirEntreContas() {

        Conta c1 = repository.save(new Conta("João", 1000.0));
        Conta c2 = repository.save(new Conta("Maria", 500.0));

        service.transferir(c1.getId(), c2.getId(), 200.0);

        Conta origem = repository.findById(c1.getId()).get();
        Conta destino = repository.findById(c2.getId()).get();

        assertEquals(800.0, origem.getSaldo());
        assertEquals(700.0, destino.getSaldo());
    }

    @Test
    public void naoDeveTransferirSaldoInsuficiente() {

        Conta c1 = repository.save(new Conta("João", 100.0));
        Conta c2 = repository.save(new Conta("Maria", 500.0));
```

```
        assertThrows(RuntimeException.class, () -> {
            service.transferir(c1.getId(), c2.getId(), 200.0);
        });
    }
}
```

application.properties

```
spring.datasource.url=jdbc:h2:mem:testdb
spring.jpa.hibernate.ddl-auto=create-drop
```

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Comparação com seu exemplo anterior

JDBC puro	Spring Boot
Muito código	Menos código
Controle manual	Automático
Didático	Produtivo